

Proyecto 2: Divide y vencerás

Felipe Paillacar[†], Constanza Araya[†] y Guillermo Vargas[†]

¹Docente: Jaqueline Aldridge Aguila

²Diseño de algoritmos

³Repositorio: <https://github.com/FelipePaillacar/Algoritmos-2/tree/main>

[†]Universidad de Magallanes

15 de mayo de 2026

Resumen

Este proyecto aborda la implementación y el análisis experimental de algoritmos basados en la estrategia de divide y vencerás integrados a un sistema de gestión de deportistas. El estudio se centra en evaluar el rendimiento de técnicas de ordenamiento como Merge Sort (con optimización de umbral) y Quick Sort (con diversas variantes de pivote), junto con métodos de búsqueda binaria y exponencial. El trabajo integra un análisis de complejidad teórica contrastado con pruebas empíricas de tiempo de ejecución para determinar la eficiencia de cada algoritmo.

■ Índice

1	Objetivos	3
2	Introduccion	4
3	Conceptos teoricos	5
3.1	Divide y venceras	5
3.2	Complejidad de un algoritmo	5
	Conceptos clave de complejidad (notacion Big O)	
3.3	Esquema de particion de Lomuto	5
3.4	Interpolacion	5
4	Algoritmos de ordenamiento	6
4.1	Merge Sort	6
4.2	Merge Sort con Insertion Sort	6
4.3	Quick Sort	6
5	Algoritmos de busqueda	8
5.1	Busqueda binaria	8
5.2	Busqueda binaria recursiva	8
	Diferencias clave en ejecucion busqueda binaria iterativa y recursiva	
5.3	Complejidad temporal busqueda binaria iterativa y recursiva	8
5.4	Busqueda binaria para rangos	9
	Complejidad temporal busqueda binaria para rangos	
5.5	Busqueda exponencial	10
	Funcionamiento de la busqueda exponencial • Complejidad temporal	
5.6	Busqueda por interpolacion	10
	Complejidad temporal	
6	Algoritmos de seleccion	12
6.1	Quick Select	12
	Complejidad temporal:	
7	Analisis de casos: ordenamiento	13
7.1	Merge Sort y Merge Sort con Insertion Sort	13
	Mejor caso • Peor caso • Caso promedio	
7.2	Quick Sort	14
	Mejor caso • Peor caso • Caso promedio	

8	Análisis de casos: búsquedas	17
8.1	Mejor caso	17
8.2	Peor caso	17
8.3	Caso promedio	18
9	Análisis de casos: Quick select	19
9.1	Comparativa de casos	19
10	Análisis de casos: tarea 1 vs tarea 2	20
10.1	Mejor caso	20
10.2	Peor caso	20
10.3	Caso promedio	21
11	Decisiones del código	22
11.1	Ranking	22
12	Conclusion	23
13	Contribuciones	24

1. Objetivos

- Implementar correctamente algoritmos utilizando la estrategia de divide y vencerás.
- Analizar empíricamente la complejidad y el rendimiento de los algoritmos implementados, evaluando además el impacto de sus distintas optimizaciones y variantes.
- Documentar adecuadamente el código desarrollado y los resultados obtenidos del análisis.

2. Introduccion

En el siguiente informe se analizarán los resultados obtenidos mediante la implementación de distintos algoritmos de búsqueda, ordenamiento y selección, los cuales se fundamentan en el enfoque de "divide y vencerás". Dichos algoritmos fueron desarrollados en el lenguaje C y puestos a prueba utilizando datos de deportistas generados de forma aleatoria. Se realizaron mediciones de los tiempos de ejecución para cada método con el objetivo de evaluar su eficiencia empírica y contrastar estos hallazgos con los resultados teóricos de la complejidad computacional

3. Conceptos teoricos

3.1. Divide y venceras

Divide y venceras es una estrategia algoritmica. Un algoritmo divide y venceras tipico resuelve un problema siguiendo tres pasos:

- **Dividir:** Descompones el problema en subproblemas mas pequeños, cada subproblema debe representar una parte del problema original, Por lo general este paso emplea un enfoque recursivo para dividir el problema hasta que no es posible crear subproblemas mas pequeños.
- **Vencer:** Resolver los sub-problemas recursivamente. Esto se conoce como el caso base, es decir el subproblema es tan pequeño que se 'resolvio solo'.
- **Combinar:** Combinar apropiadamente las soluciones de los subproblemas para obtener la solucion del problema original.

3.2. Complejidad de un algoritmo

La complejidad es una medida de que tan eficiente es el algoritmo para resolver un problema, es decir es una medida de cuanto tiempo y espacio (memoria) requiere el algoritmo para producir una solucion. La complejidad de un algoritmo se expresa utilizando la notacion **Big O**. Esta notacion proporciona una forma de comparar la complejidad de distintos algoritmos. Por ejemplo, un algoritmo con una complejidad temporal de $O(n)$ se considera más eficiente que un algoritmo con una complejidad temporal de $O(n^2)$, porque crece a un ritmo más lento a medida que aumenta el tamaño de entrada. En general, el objetivo del diseño de algoritmos es encontrar algoritmos que sean eficientes y fáciles de implementar. Esto implica un compromiso entre la complejidad del tiempo y el espacio, así como otros factores como la simplicidad (facil entendimiento) y la mantenibilidad (facilidad de actualización, corrección o mejora).

3.2.1. Conceptos clave de complejidad (notacion Big O)

Es comun confundir la notacion Big O como una medida de tiempo, pero la realidad es que la notacion Big O nos indica cuantas operaciones debe realizar un algoritmo para resolver un problema, ya que si fuera una medida de tiempo podria variar dependiendo del hardware en el que se estan realizando las pruebas. Por ello se habla del mejor y peor caso, es decir el mejor caso o caso ideal seria cuando un algoritmo tiene que hacer operaciones minimas para llegar a la solucion. Por ejemplo, en el caso de la busqueda lineal, el mejor caso es cuando el elemento que se busca esta en la primera posicion del arreglo, por lo que solo se realiza una operacion. A continuacion se muestra una comparacion de las funciones de complejidad mas comunes.

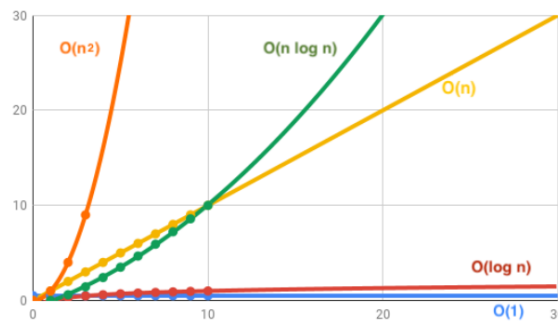


Figura 1. Comparacion de las funciones de complejidad mas comunes.

- $O(1)$: Tiempo constante. El tiempo de ejecución no cambia independiente del tamaño de la entrada.
- $O(\log n)$: Tiempo logarítmico, muy eficiente. El tiempo de ejecución crece lentamente a medida que aumenta el tamaño de la entrada.
- $O(n)$: Tiempo lineal. El tiempo de ejecución crece proporcionalmente al tamaño de la entrada.
- $O(n \log n)$: Tiempo log-lineal. Es casi una línea recta, pero con una ligera inclinación hacia arriba.
- $O(n^2)$: Tiempo cuadrático. El tiempo de ejecución crece proporcionalmente al cuadrado del tamaño de la entrada.

3.3. Esquema de particion de Lomuto

El esquema de particion de Lomuto es un algoritmo popular utilizado para dividir una matriz. Lomuto funciona seleccionando un elemento de pivote, generalmente el ultimo elemento del array, y luego dividiendolo en dos subarrays de modo que los elementos menores o iguales al pivote esten en el lado izquierdo del pivote y los elementos mayores queden a su derecha. El esquema de particion de Lomuto funciona de la siguiente manera:

- Se selecciona un elemento del array como pivote (generalmente el ultimo).
- Se recorre la lista comparando cada elemento con el pivote. Los menores se mueven a la izquierda y los mayores a la derecha.
- Al final del recorrido, se coloca el pivote en su lugar definitivo (entre las dos sublistas)
- Se repite el mismo proceso en la sublista izquierda y en la derecha hasta que todos los elementos estén ordenados.

3.4. Interpolacion

La interpolación de datos es un método matemático y estadístico utilizado para estimar valores desconocidos dentro de un rango de datos conocidos. En otras palabras, la interpolación permite calcular un valor intermedio entre dos puntos o entre una serie de puntos ya registrados. Por ejemplo si se busca un valor 15 y sabemos que el 10 esta en el indice 5 y el 20 en el indice 15, por la interpolacion se asume que el valor que se busca esta cerca de la posicion 10

4. Algoritmos de ordenamiento

Los algoritmos de ordenamiento son un conjunto de procedimientos que organizan los elementos de una lista o arreglo en un orden específico, ya sea ascendente o descendente. Los algoritmos de ordenamiento son importantes ya que facilitan la búsqueda, la organización y el análisis de datos. Son fundamentales en la eficiencia informática ya que el desorden es costoso en términos de tiempo y recursos. Algunas razones de su importancia incluyen:

■ Optimización de la búsqueda

- **Busqueda lineal:** Si los datos están desordenados, se debe revisar uno por uno hasta encontrar el elemento deseado (complejidad $O(n)$). En cambio, si los datos están ordenados, se pueden utilizar algoritmos de búsqueda más eficientes.
- **Busqueda binaria:** Este algoritmo requiere que los datos estén ordenados para funcionar correctamente. Permite encontrar un elemento en tiempo logarítmico $O(\log n)$.

■ Organización y visualización:

El ordenamiento permite agrupar elementos similares, lo cual es vital para:

- Eliminar duplicados: Es más fácil notar que un elemento está duplicado si está junto a su par.
- Priorización: En sistemas operativos o servidores, ordenar procesos por prioridad asegura que lo más importante se ejecute primero.

Información

Antes de elegir un algoritmo de ordenamiento es importante hacer algunas preguntas: ¿Cuán grande son los datos que se ordenan? ¿Cuánta memoria hay disponible? ¿Los datos crecerán?

4.1. Merge Sort

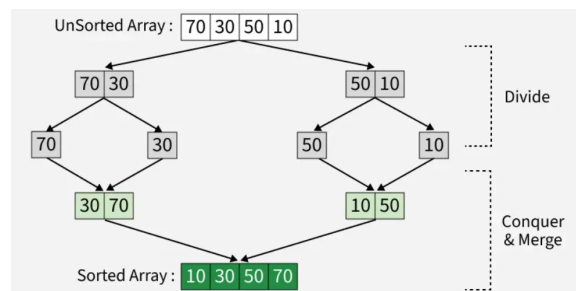


Figura 2. Merge sort no adaptativo

Merge sort es un algoritmo de ordenamiento conocido por su eficiencia y estabilidad. Utiliza la estrategia de divide y vencerás para ordenar una lista o arreglo. Funciona dividiendo recursivamente el arreglo en mitades hasta que cada subarreglo contiene un solo elemento, luego combina esos subarreglos de manera ordenada para formar el arreglo final ordenado. Merge sort es un algoritmo no adaptativo, lo que significa que su rendimiento no mejora si el array está previamente ordenado. Debido a esto su complejidad en el peor y mejor caso es $O(n \log n)$.

4.2. Merge Sort con Insertion Sort

Como se mencionó anteriormente en 4.1, merge sort es un algoritmo no adaptativo, lo que significa que su rendimiento no mejora si el array está previamente ordenado. Para ello se puede optimizar utilizando insertion sort para ordenar subarreglos pequeños, ya que insertion sort es un algoritmo adaptativo, lo que significa que su rendimiento mejora si el array está previamente ordenado, es decir Insertion sort no es 'ciego' como merge sort. Esto ayuda a que en teoría en el peor caso se mantenga la complejidad de merge sort $O(n \log n)$, pero en el mejor caso se mejora a $O(n)$, ya que insertion sort detecta que los subarreglos están ordenados y como los subarreglos son pequeños, mejora significativamente el tiempo de ejecución.

4.3. Quick Sort

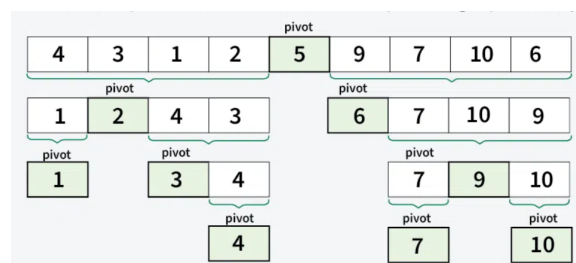


Figura 3. Quick sort

Quick sort es un algoritmo de ordenamiento que utiliza la estrategia de divide y venceras que elige un elemento como pivote y divide el array al rededor del pivote, como se nombro en [3.3](#). Este algoritmo utiliza recursividad, para ello es importante tener claro cual es el caso base, el cual es cuando el array tiene un solo elemento o esta vacio, ya que en ese caso el array ya esta ordenado. Hay diferentes formas de elegir un pivote, como por ejemplo:

- **Pivote primer elemento**
- **Pivote ultimo elemento**
- **Pivote aleatorio**
- **Pivote mediana de tres**

La eleccion del pivote es importante ya que puede afectar significativamente el rendimiento del algoritmo. Lo cual sera analizado en el transcurso de este proyecto.

5. Algoritmos de búsqueda

Los algoritmos de búsqueda funcionan mediante un método paso a paso para localizar datos específicos entre un conjunto de datos. Son métodos computacionales para localizar un elemento específico dentro de un conjunto de datos, devolviendo su posición o confirmando su existencia.

5.1. Búsqueda binaria

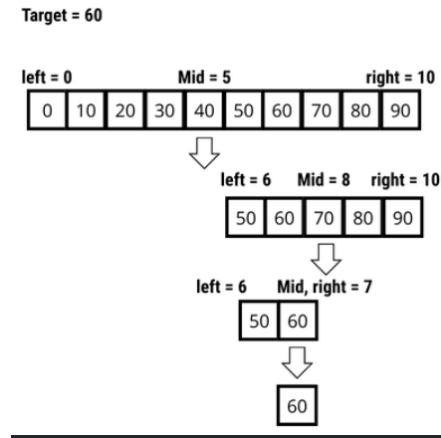


Figura 4. Búsqueda binaria

La búsqueda binaria es un algoritmo que se basa en el principio de 'divide y vencerás'. Permite encontrar elementos en una lista ordenada mucho más rápido que la búsqueda lineal (bajo ciertas condiciones), esto se debe a que la búsqueda binaria reduce el espacio de búsqueda a la mitad en cada iteración, hasta llegar a la solución o determinar que el elemento no está presente. Se llama binaria porque en cada paso se divide el espacio de búsqueda en dos partes iguales. La búsqueda binaria es más eficiente que la búsqueda lineal bajo ciertas condiciones, tales como:

- **Datos ordenados:** Esta es la condición obligatoria para que la búsqueda binaria funcione, los datos deben estar ordenados (ascendente o descendente). Si la lista está desordenada, la búsqueda binaria no funcionará correctamente y se tendrá que utilizar la búsqueda lineal.
- **Grandes volúmenes de datos:** La diferencia se vuelve más significativa a medida que el tamaño de la lista aumenta. Para listas pequeñas, la búsqueda lineal puede ser más rápida.

5.2. Búsqueda binaria recursiva

La búsqueda binaria recursiva es una implementación de la búsqueda binaria que utiliza la recursividad para dividir el espacio de búsqueda. Funciona de la siguiente manera:

- **Caso base 1:** Si el límite izquierdo supera al derecho, el elemento no está en la lista
- **Caso base 2:** Si el elemento del medio es igual al elemento buscado, se devuelve la posición del elemento (elemento encontrado)
- **Caso recursion**
 - Si el elemento es menor que el elemento medio, se llama a la función para la mitad izquierda.
 - Si el elemento es mayor que el elemento medio, se llama a la función para la mitad derecha.

5.2.1. Diferencias clave en ejecución búsqueda binaria iterativa y recursiva

- La versión iterativa es generalmente más eficiente para sistemas con recursos limitados, ya que no consume memoria adicional, pero visualmente el código puede verse más 'sucio'
- La versión recursiva consume más memoria RAM, debido a que realiza llamadas recursivas y en cada nueva llamada se crea un nuevo marco de la pila de llamadas (stack).

5.3. Complejidad temporal búsqueda binaria iterativa y recursiva

Teóricamente en el caso iterativo y recursivo se mantiene la misma complejidad temporal de $O(\log n)$, esto se deduce de la siguiente manera para el peor caso:

1. Inicio: Se tiene una lista con n elementos
2. Paso 1: Se divide la lista en dos partes iguales, es decir $\frac{n}{2}$
3. Paso 2: Se divide nuevamente la mitad seleccionada en dos partes, es decir $\frac{n}{2} \cdot \frac{1}{2} = \frac{n}{4}$
4. Paso 3: Se divide nuevamente la mitad seleccionada en dos partes, es decir $\frac{n}{4} \cdot \frac{1}{2} = \frac{n}{8}$
5. Paso k: Queda $\frac{n}{2^k}$ elementos

El algoritmo se detiene en el peor de los casos cuando solo queda un elemento por revisar, de el analisis anterior ($\frac{n}{2^k}$) podemos igualar cuando solo queda un elemento, es decir:

$$\frac{n}{2^k} = 1$$

Luego despejando k, se obtiene:

$$k = \log_2(n)$$

Por lo tanto, el numero maximo de comparaciones es proporcional al logaritmo del numero de elementos, lo que se expresa como $O(\log n)$. Luego para el mejor caso, el cual ocurre cuando el elemento que se busca esta exactamente en la mitad del array en la primera comparacion, como solo se realiza una comparacion independientemente del tamaño del array, se tiene una complejidad de $O(1)$. Cabe recalcar que todos estos analisis son teoricos, los cuales se respaldaran con pruebas empiricas de tiempo de ejecución para determinar la eficiencia de cada algoritmo.

Información

La busqueda recursiva es mas facil de entender y escribir, pero puede consumir mas memoria debido a las llamadas recursivas. La busqueda iterativa es mas eficiente en terminos de memoria, pero puede ser menos intuitiva para algunos programadores.

5.4. Busqueda binaria para rangos

La busqueda binaria para rangos es una variante del algoritmo clasico, la cual se utiliza cuando no solo se quiere saber si un elemento existe, si no que se quiere encontrar un intervalo de elementos que cumplen con una condicion especifica o son identicos.

En lugar de detenerse apenas encuentra el valor, el algoritmo sigue buscando hacia los extremos para delimitar el 'rango' donde se repite ese valor o donde se cumplen ciertas condiciones.

Se utiliza principalmente para responder dos tipos de preguntas en un arreglo ordenado:

- Contar repeticiones: ¿Cuántas veces se repite un valor específico en el arreglo?
- Encontrar limites: ¿Cuáles son los indices de todos los elementos que estan entre un rango de valores específicos?

Proceso logico

En lugar de recorrer toda la lista, el algoritmo hace dos busquedas rapidas, por ejemplo si se quiere encontrar el rango 10 – 50:

- **Busqueda del limite inferior:** Se busca el numero 10. Si el 10 no esta presente, se busca el primer numero que sea mayor que 10 (por ejemplo 11). El algoritmo devuelve ese indice (indice inicio=4).
- **Busqueda del limite superior:** Se busca el numero 50. Si el 50 no esta presente, se busca el primer numero que sea menor que 50 (por ejemplo 49). El algoritmo devuelve ese indice (indice final=50).

Una vez que se tienen ambos indices, ya se sabe exactamente donde comienza y termina dicho rango, sin tener que mirar los elementos de en medio, ya que se sabe que todos los elementos que estan entre esos indices cumplen con la condicion de estar entre 10 y 50 porque la lista ya estaba ordenada.

Nota

La busqueda binaria para rangos es especialmente util en sistemas cuando se filtran productor por precio, cuando se buscan archivos por fecha, etc.

5.4.1. Complejidad temporal busqueda binaria para rangos

Peor caso:

Para encontrar un rango (un limite inferior y superior), se realizan dos busquedas binarias independientes. Teoricamente esto se calcula como:

$$O(\log n) + O(\log n) = 2 \cdot O(\log n)$$

como en analisis de algoritmos se ignoran las constantes, se tiene una complejidad temporal de $O(\log n)$ para encontrar un rango utilizando busqueda binaria para rangos en el peor caso.

Información

En el análisis de algoritmos e ingeniería, se ignoran las constantes porque lo que interesa es el comportamiento a largo plazo cuando el volumen de datos (n) se vuelve muy grande, pero para casos pequeños, las constantes pueden tener un impacto significativo en el rendimiento real del algoritmo.

Mejor caso:

El mejor caso ocurre cuando tanto el limite inferior como el superior se encuentran en la primera comparacion de sus respectivas busquedas

- **Busqueda del limite inferior:** El valor buscado esta justo en el medio del arreglo y el elemento anterior a el es diferente
- **Busqueda del limite superior:** El valor buscado esta justo en el medio del arreglo y el elemento siguiente a el es diferente

Debido a esto es de orden $O(1)$, ya que al igual que la busqueda binaria simple, si los elementos que definen el inicio y el final del rango estan en las posiciones que el algoritmo revisa primero, el tiempo de ejecucion no depende del tamaño del arreglo

5.5. Búsqueda exponencial

El algoritmo de búsqueda exponencial se centra en un rango de un array de entrada donde asume que debe estar presente el elemento requerido, y realiza una búsqueda binaria en ese pequeño rango. Este algoritmo también se conoce como búsqueda por duplicación o búsqueda de dedos.

Este algoritmo aumenta exponencialmente en potencias de dos, esto debido a que busca el elemento dando saltos cada vez mayores, es decir, primero revisa el elemento en la posición 1, luego en la posición 2, luego en la posición 4, luego en la posición 8, y así sucesivamente, hasta que encuentra un elemento que es mayor que el elemento buscado o hasta que se sale del rango del array. Una vez que se encuentra el elemento mayor que el buscado, se realiza una búsqueda binaria en el rango identificado.

5.5.1. Funcionamiento de la búsqueda exponencial

En el algoritmo de búsqueda exponencial, el salto comienza desde el primer índice del array. Por lo tanto, comparamos manualmente el primer elemento como primer paso del algoritmo.

- **Paso 1:** Se compara el elemento en la posición 1 con el elemento buscado. Si coinciden, se devuelve la posición 1.
- **Paso 2:** Si el elemento no coincide, se compara el elemento con el siguiente índice (2,4,8,...) y así sucesivamente, hasta que se encuentra un elemento que es mayor que el elemento buscado o hasta que se sale del rango del array.
- **Paso 3:** Cuando se sabe que el elemento buscado está entre el último salto del array y el penúltimo, se aplica búsqueda binaria únicamente en ese intervalo.

5.5.2. Complejidad temporal

Mejor caso:

El mejor caso ocurre cuando el elemento buscado está en la primera posición del array `array[0]`, ya que en ese caso cae en la comprobación **if** (`arreglo[0] == objetivo`) **return 0**; como se realiza una sola comparación, la complejidad temporal en el mejor caso es $O(1)$.

Peor caso:

En el peor de los casos, tenemos que recorrer el arreglo hasta encontrar el rango donde está el dato. Sin embargo, como los saltos son cada vez más grandes (1, 2, 4, 8,...), llegamos a posiciones muy lejanas en muy pocos pasos. Por eso, aunque el dato esté muy al final, el tiempo que tarda no crece de forma pesada, sino que se mantiene eficiente porque solo trabaja con la parte del arreglo donde realmente está el objetivo.

Es por esto que la complejidad está dada por $O(\log i)$, esto se debe a que hay que dar k saltos para encontrar la posición i , de la siguiente manera:

$$2^{\text{saltos}(k)} = i$$

Aplicando logaritmo en ambos lados, se obtiene:

$$k = \log_2(i)$$

Luego, una vez que se encuentra el rango donde está el elemento buscado, se realiza una búsqueda binaria en dicho rango, de ?? se sabe que la búsqueda binaria tiene una complejidad de $O(\log n)$, pero este intervalo es más pequeño que el original, es por eso que se expresa como $O(\log \frac{i}{2})$, por lo tanto, la complejidad total del algoritmo de búsqueda exponencial en el peor caso es la suma de la búsqueda del rango ($O(\log i)$) y la búsqueda binaria ($O(\log \frac{i}{2})$), lo que se expresa como:

$$O(\log i) + O(\log \frac{i}{2})$$

Por propiedades de los logaritmos:

$$O(\log \frac{i}{2}) = O(\log i - \log 2) = O(\log i) - 1$$

Reemplazando en la expresión original, se obtiene:

$$O(\log i) + O(\log i) - 1 = 2 \cdot O(\log i) - 1$$

Como se mencionó anteriormente en 5.4.1, en el análisis de algoritmos se ignoran las constantes para datos muy grandes, por lo tanto, la complejidad temporal del algoritmo de búsqueda exponencial en el peor caso es $O(\log i)$.

5.6. Búsqueda por interpolación

Esta búsqueda aplica la interpolación, de 3.4 sabemos que la interpolación es un método matemático que se utiliza para estimar valores desconocidos dentro de un rango de datos conocidos.

La búsqueda por interpolación es una mejora sobre la búsqueda binaria para instancias, donde los valores en un arreglo ordenado están distribuidos uniformemente. La interpolación construye nuevos puntos de datos dentro del rango de un conjunto discreto de puntos de datos conocidos. La búsqueda binaria siempre va al elemento central para verificar. Por otro lado, la búsqueda por interpolación puede ir a diferentes ubicaciones según el valor de la clave que se está buscando. Por ejemplo, si el valor de la clave está más cerca del último elemento, es probable que la búsqueda por interpolación comience la búsqueda hacia el lado del final.

Es similar a cuando buscas una palabra en un diccionario, si se busca la palabra 'Árbol', no se abre el libro por la mitad, sino que se va directo a las primeras páginas. Del mismo modo si se busca la palabra "Zapato", se vadiectamente al final.

Calculo de la posición

La posición se calcula utilizando la siguiente fórmula:

$$pos = low + \frac{(target - arr[low]) \cdot (high - low)}{arr[high] - arr[low]}$$

Donde:

- *low* es el índice del primer elemento del arreglo.
- *high* es el índice del último elemento del arreglo.
- *arr[low]* es el valor del primer elemento del arreglo.
- *arr[high]* es el valor del último elemento del arreglo.
- *target* es el valor que se está buscando.

Ejemplo:

Si se tiene un array de 10 elementos: array[10,20,30,40,50,60,70,80,90,100] y se busca el valor 80, se puede aplicar la formula de la siguiente manera:

- *x*=80 (lo que buscamos)
- *low*=0 (índice del primer elemento)
- *high*=9 (índice del ultimo elemento)
- *arr[low]*=10 (valor del primer elemento)
- *arr[high]*=100 (valor del ultimo elemento)

Aplicando la formula, se obtiene:

$$pos = 0 + \frac{(80 - 10) \cdot (9 - 0)}{100 - 10}$$

$$pos = 7$$

Por lo tanto, la posición estimada del valor 80 es el índice 7, lo que es correcto, ya que el valor 80 se encuentra en esa posición del arreglo.

5.6.1. Complejidad temporal**Caso promedio:**

El caso promedio ocurre cuando los elementos del arreglo están distribuidos uniformemente (como en el ejemplo anterior), en este caso, la búsqueda por interpolación tiene una complejidad de $O(\log \log n)$

Peor caso:

El peor caso ocurre cuando los elementos del arreglo no están distribuidos uniformemente, por ejemplo si se tiene un arreglo con elementos donde la mayoría son números pequeños y solo hay un número grande, como por ejemplo: array[1,2,4,6,100], si se busca el número 2, debido al valor del último elemento (100) la fórmula de interpolación dará una posición imprecisa, es por esto que en el peor caso la búsqueda por interpolación tiene una complejidad de $O(n)$

6. Algoritmos de seleccion

Un algoritmo de selección es un método diseñado para encontrar el k -ésimo elemento más pequeño (o más grande) en una lista o arreglo desordenado. A diferencia de los algoritmos de ordenamiento, que organizan toda la lista, la selección se enfoca en localizar un solo valor específico basado en su posición.

6.1. Quick Select

QuickSelect es un algoritmo de selección diseñado para encontrar el k -ésimo elemento más pequeño en una lista desordenada. Su funcionamiento se basa en encontrar un punto pivote: una posición que divide la lista en dos partes, donde cada elemento a la izquierda es menor o igual que el pivote y cada elemento a la derecha es mayor. Una vez realizada la partición, se compara el índice del pivote con k :

- Si el índice del pivote es igual a k , se ha encontrado el elemento y se devuelve.
- Si el índice es mayor que k , el algoritmo se repite recursivamente solo para la parte izquierda.
- Si el índice es menor que k , el algoritmo se repite para la parte derecha.

Existen dos esquemas clásicos para realizar esta partición:

- Partición de Lomuto.
- Partición de Hoare.

En este caso, se trabajará con la **partición de Lomuto**, detallada en la sección 3.3. Este esquema elige generalmente el último elemento como pivote y mantiene un índice i mientras recorre el arreglo con un índice j . De esta forma, los elementos desde el inicio hasta $i - 1$ son menores o iguales al pivote, mientras que los elementos desde i hasta $j - 1$ son mayores que este.

6.1.1. Complejidad temporal:

Caso promedio:

En el caso promedio, QuickSelect tiene una complejidad de $O(n)$, esto se debe a que cada partición divide el arreglo en dos partes aproximadamente iguales, descartando la mitad de los elementos en cada paso. Por lo tanto, el número de elementos a revisar se reduce significativamente en cada iteración.

Peor caso:

El peor caso ocurre cuando hay una mala elección del pivote, por ejemplo, si el pivote es siempre el elemento más grande o más pequeño del arreglo, lo que resulta en una partición muy desequilibrada. En este caso es de orden $O(n^2)$.

7. Analisis de casos: ordenamiento

7.1. Merge Sort y Merge Sort con Insertion Sort

7.1.1. Mejor caso

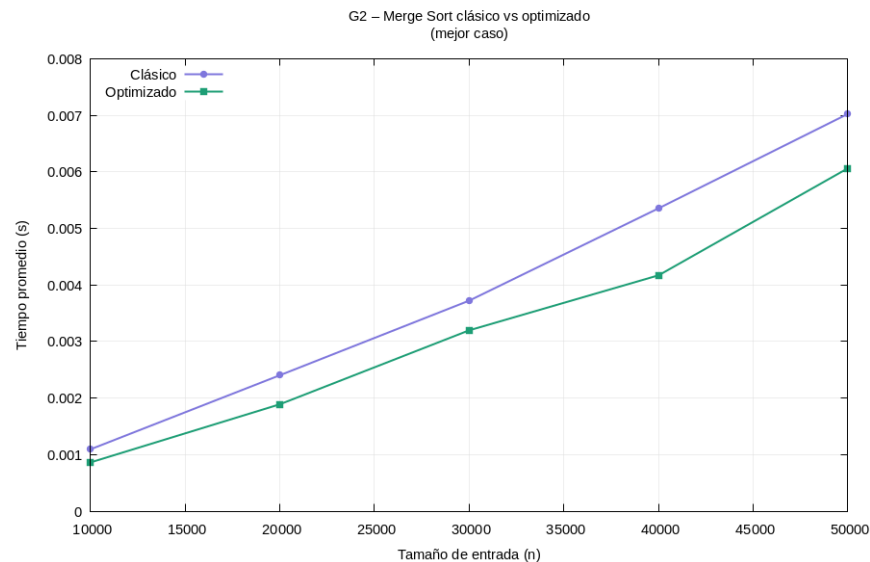


Figura 5. Merge sort vs Merge sort con Insertion sort mejor caso

Se puede observar que en el mejor caso, Merge Sort con Insertion Sort presenta un tiempo de ejecución notablemente inferior al Merge Sort tradicional. Esto se debe a que el Insertion Sort es un algoritmo adaptativo: su rendimiento mejora drásticamente cuando los datos ya están ordenados. A diferencia del Merge Sort convencional, que es 'ciego' y realiza todas las divisiones y comparaciones independientemente del orden de los datos, el Insertion Sort simplemente verifica que el elemento ya está en su posición, reduciendo el trabajo a $O(n)$ en los sub-arreglos finales. Esto permite que la versión optimizada aproveche la estructura previa del arreglo, optimizando la eficiencia de forma que el Merge tradicional no.

7.1.2. Peor caso

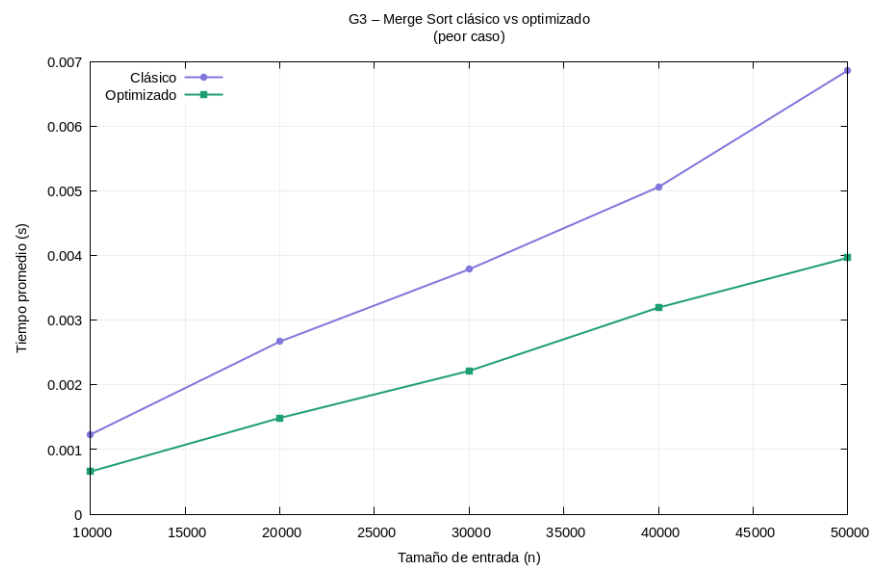


Figura 6. Merge sort vs Merge sort con Insertion sort peor caso

Analizando el gráfico se puede observar que a medida que el tamaño de la entrada aumenta, la diferencia en los tiempos de ejecución entre ambos algoritmos se vuelve más evidente.

Aunque teóricamente ambos algoritmos mantienen una complejidad de $O(n \log n)$ en el peor caso, la versión optimizada muestra tiempos menores.

Mientras que el Merge Sort tradicional se vuelve lento al realizar una cantidad inmensa de divisiones minúsculas, la versión optimizada detiene esas divisiones antes de tiempo y resuelve los grupos restantes de forma más directa. El gráfico confirma que, al evitar esa carga de trabajo innecesaria en los niveles más bajos del proceso, se obtiene una ejecución mucho más ágil y eficiente.

7.1.3. Caso promedio

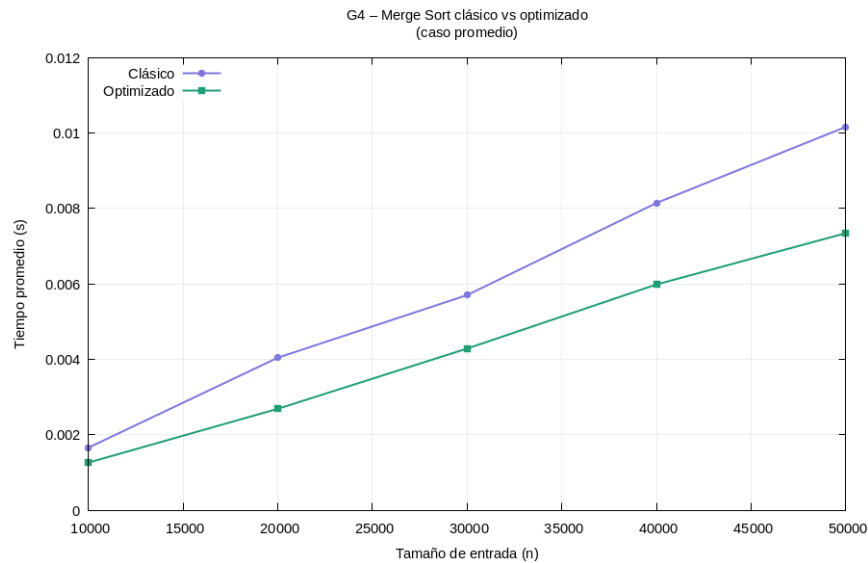


Figura 7. Merge sort vs Merge sort con Insertion sort caso promedio

En el gráfico del caso promedio se observa que, al aumentar la cantidad de datos hasta 50,000, la versión optimizada es claramente más eficiente que la clásica. Esto ocurre porque el método clásico gasta demasiado tiempo en los pasos finales de la división, realizando miles de operaciones pequeñas que suman un retraso importante. La línea verde, al detener las divisiones antes de tiempo, evita toda esa carga de trabajo innecesaria, logrando que el tiempo total de ejecución sea mucho más bajo y estable.

7.2. Quick Sort

7.2.1. Mejor caso

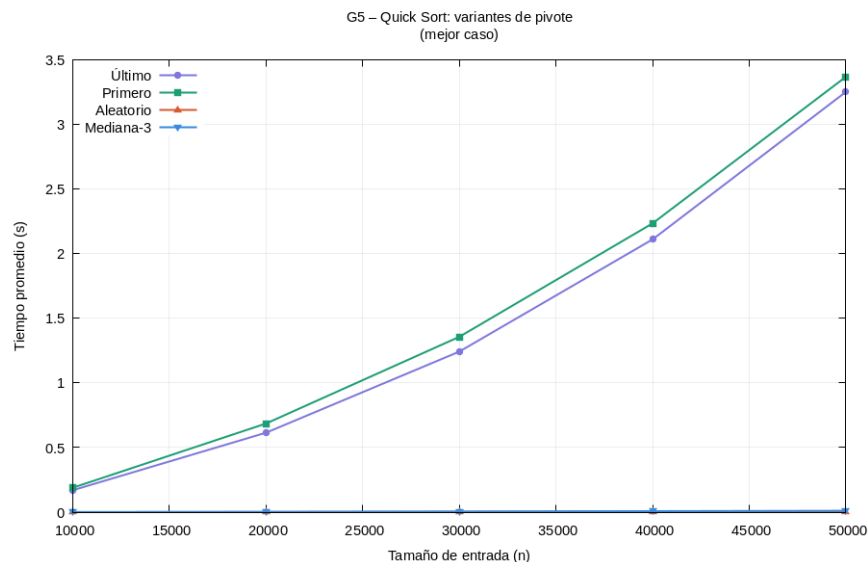


Figura 8. Quick sort mejor caso

En el gráfico del mejor caso, se observa un contraste drástico dependiendo de la elección del pivote. Las variantes de Aleatorio y Mediana muestran un rendimiento óptimo, con tiempos de ejecución cercanos a cero, lo que demuestra una división eficiente del trabajo. Por otro lado, las estrategias

de Primero y Último presentan curvas que crecen rápidamente, alcanzando tiempos significativamente mayores. Esto indica que, para este conjunto de datos, estas últimas estrategias no logran realizar una partición equilibrada, transformando lo que debería ser un caso ideal en uno mucho más pesado.

7.2.2. Peor caso

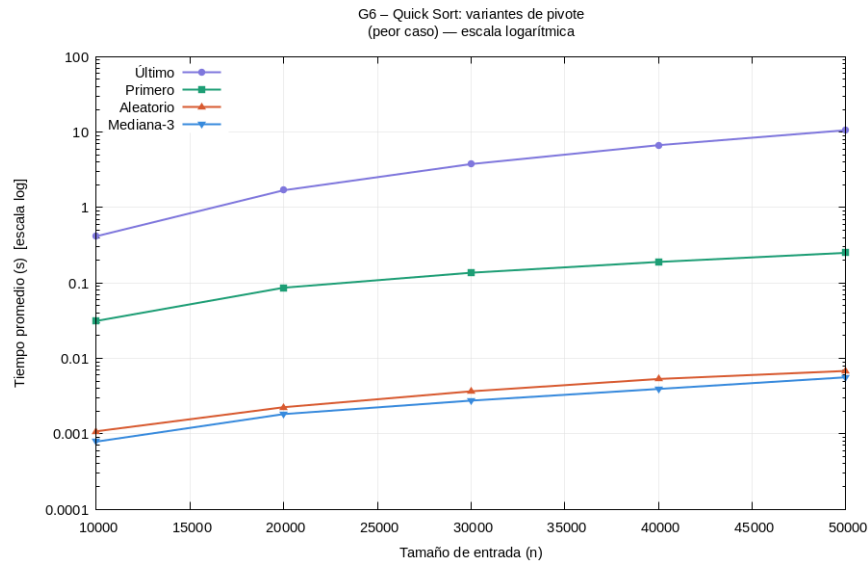


Figura 9. Quick sort peor caso

Los resultados demuestran que el rendimiento de Quick Sort es altamente sensible a la elección del pivote. Mientras que las estrategias fijas pueden hacer que el algoritmo se vuelva extremadamente lento en ciertos escenarios, las variantes de Aleatorio y Mediana garantizan una alta eficiencia constante. Esto confirma que seleccionar el pivote de forma estratégica es la mejor manera de asegurar que el programa responda rápidamente bajo cualquier condición.

7.2.3. Caso promedio

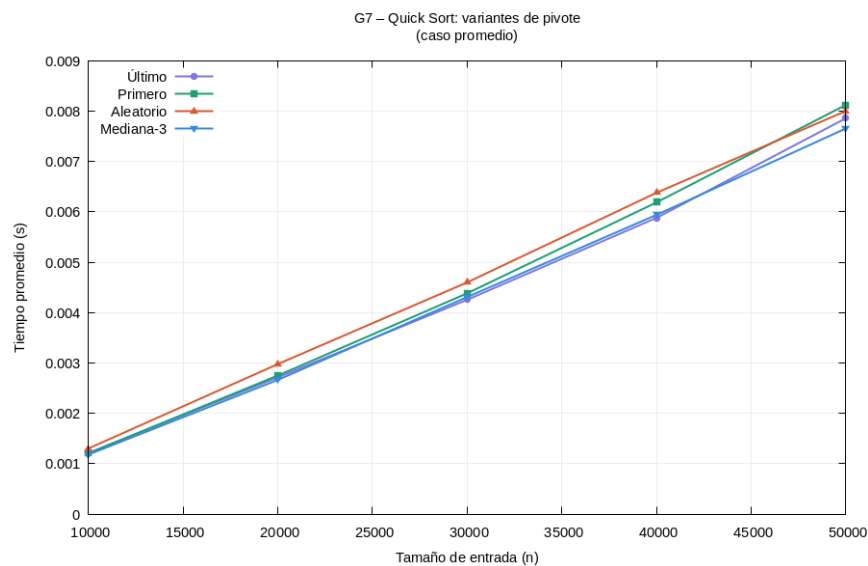


Figura 10. Quick sort caso promedio

En el caso promedio, el gráfico muestra un rendimiento muy similar entre todas las variantes de pivote. Al trabajar con datos distribuidos de forma aleatoria, el algoritmo logra realizar particiones equilibradas de manera natural, manteniendo tiempos de ejecución extremadamente bajos (cerca de los 0.008 segundos para 50,000 elementos). La ligera diferencia en la variante Aleatoria se debe al pequeño costo adicional de

procesar la elección del pivote al azar, pero en general, el gráfico confirma que Quick Sort es un algoritmo altamente eficiente para el uso cotidiano, independientemente de la estrategia de partición seleccionada.

8. Analisis de casos: busquedas

8.1. Mejor caso

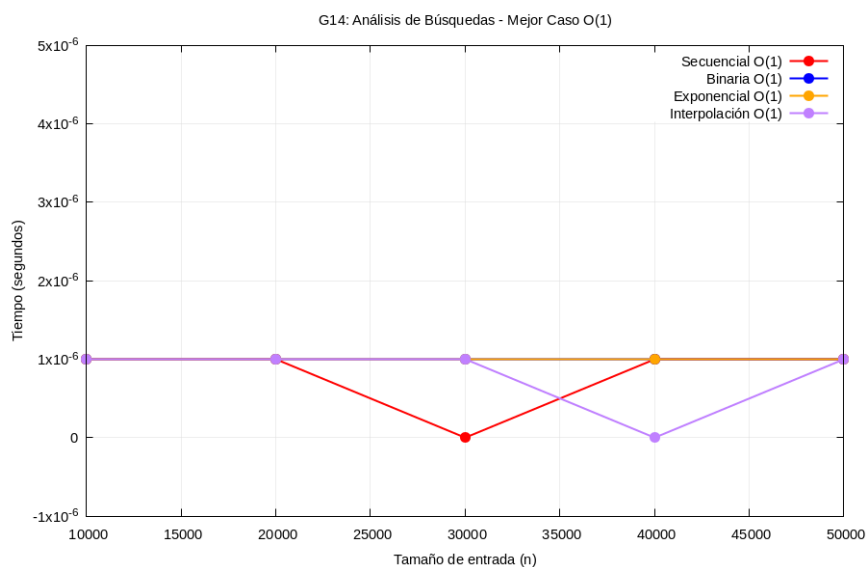


Figura 11. Búsqueda binaria iterativa vs Búsqueda binaria recursiva mejor caso

8.2. Peor caso

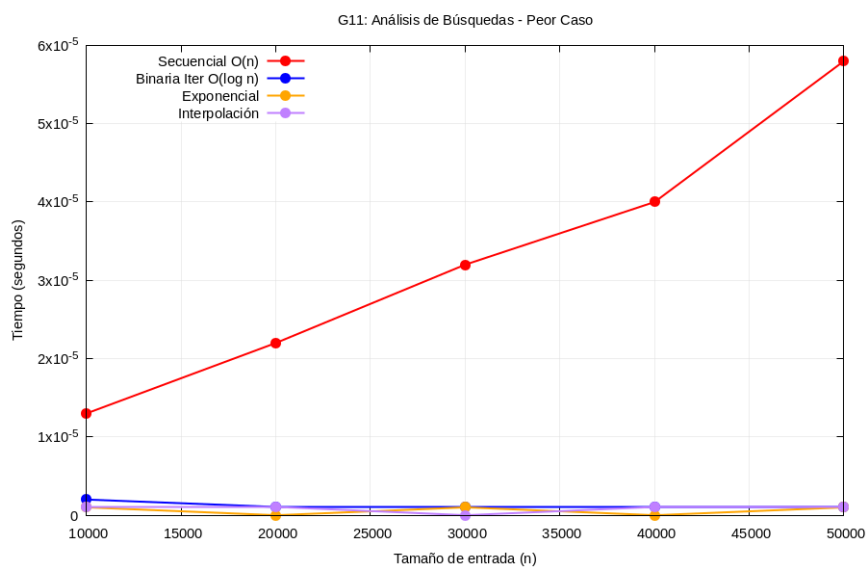


Figura 12. Búsqueda binaria iterativa vs Búsqueda binaria recursiva peor caso

8.3. Caso promedio

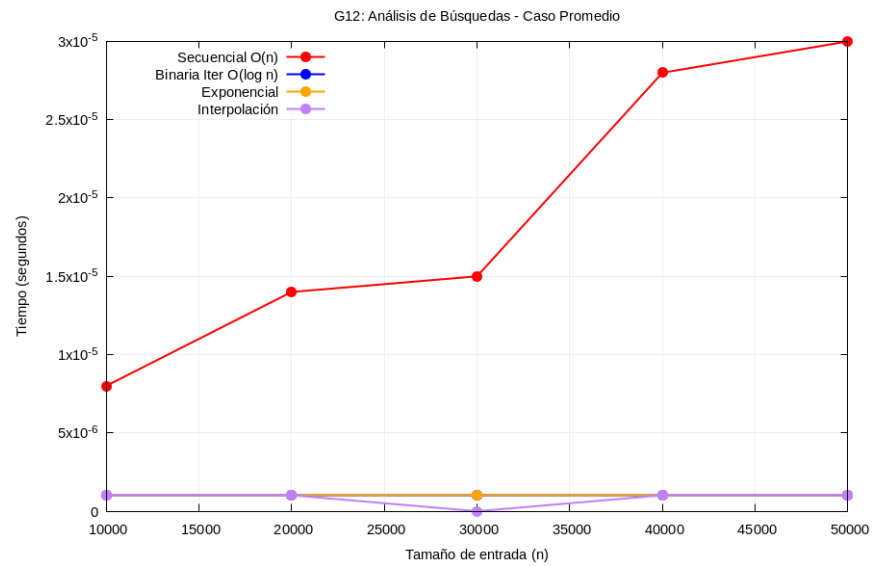


Figura 13. Búsqueda binaria iterativa vs Búsqueda binaria recursiva caso promedio

9. Analisis de casos: Quick select

9.1. Comparativa de casos

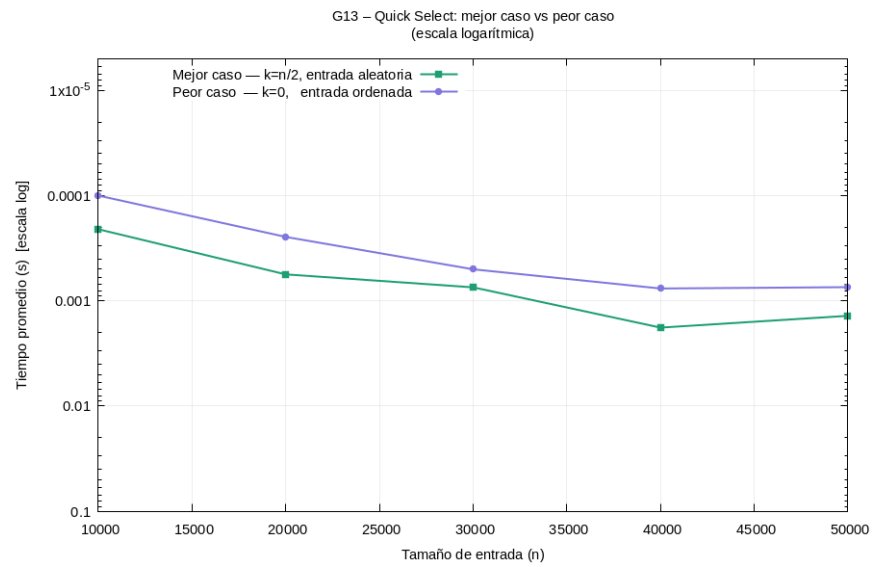


Figura 14. Quick select mejor caso vs peor caso vs caso promedio

10. Analisis de casos: tarea 1 vs tarea 2

10.1. Mejor caso

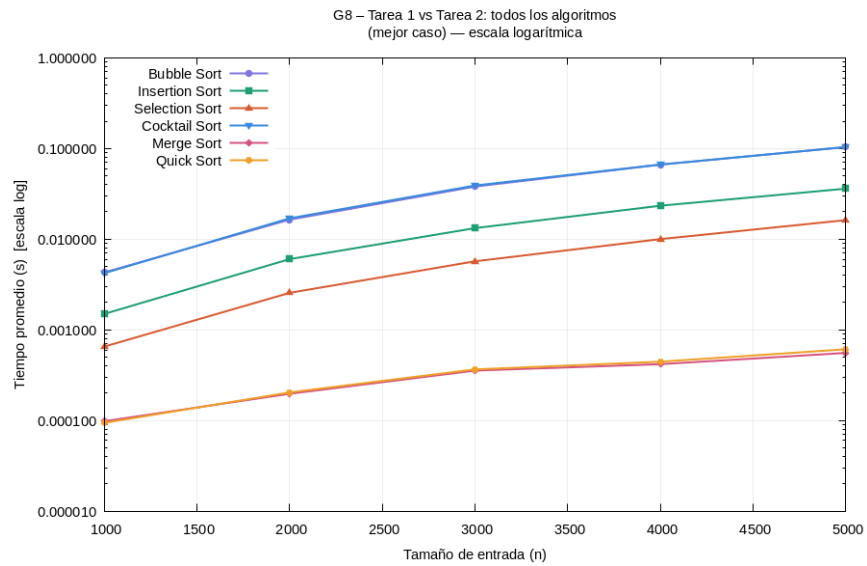


Figura 15. Tarea 1 vs Tarea 2 mejor caso

10.2. Peor caso

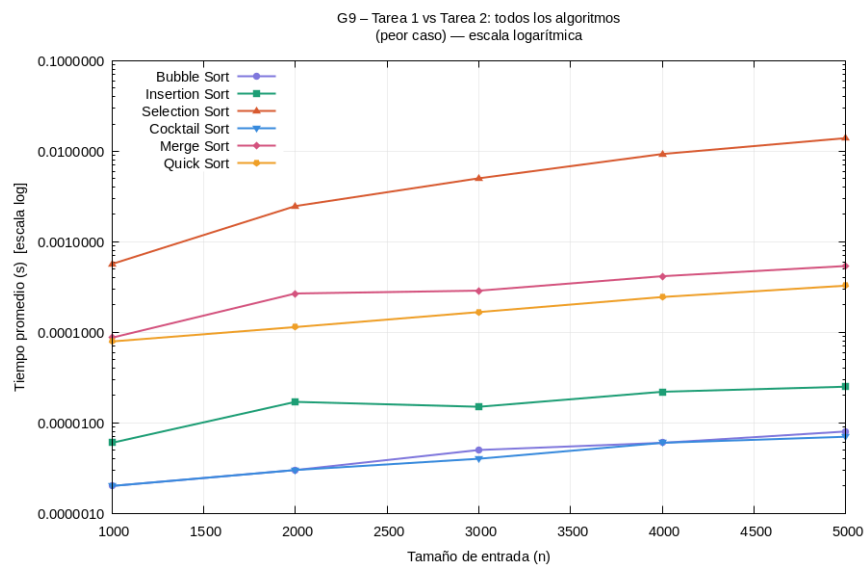


Figura 16. Tarea 1 vs Tarea 2 peor caso

10.3. Caso promedio

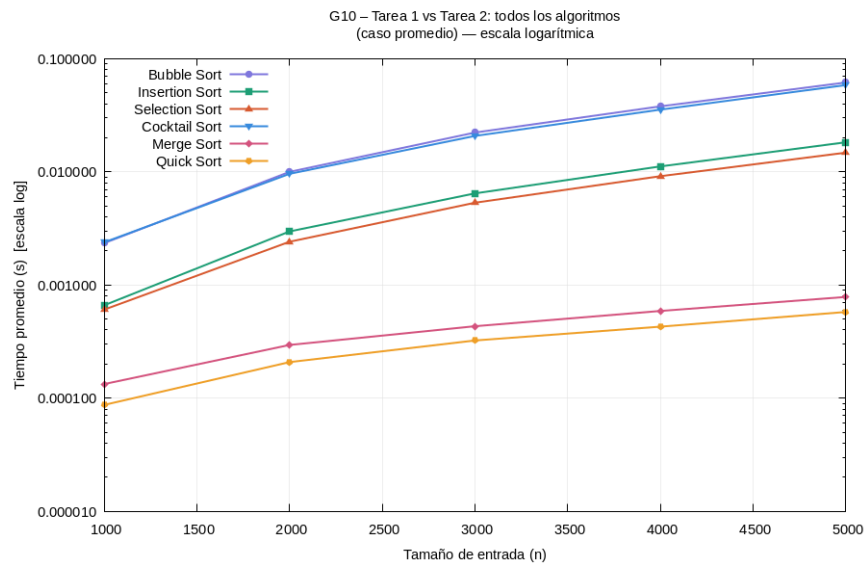


Figura 17. Tarea 1 vs Tarea 2 caso promedio

11. Decisiones del código

11.1. Ranking

Para el ranking se decidió utilizar el algoritmo Quick select, ya que es un algoritmo de selección que permite encontrar el k -ésimo elemento más pequeño de una lista sin necesidad de ordenar toda la lista, lo que lo hace muy eficiente para este tipo de tareas. A diferencia de un ordenamiento completo (como Merge Sort o Quick Sort) que tarda $O(n \log n)$, Quick Select tiene un tiempo de ejecución promedio de $O(n)$. Esto significa que, para generar un ranking o encontrar un top específico de elementos, el programa realiza mucho menos trabajo y consume menos recursos, ya que solo se enfoca en la sección que nos interesa del arreglo y descarta el resto. Es la herramienta ideal cuando el objetivo no es ver todos los datos ordenados, sino extraer posiciones clave de forma rápida.

12. Conclusion

Los experimentos realizados con hasta 50,000 datos demuestran que la eficiencia real de los algoritmos depende de su capacidad para evitar trabajo innecesario y adaptarse a la estructura de la información ya que en el caso de merge sort la versión optimizada supera a la clásica al dejar de dividir los grupos pequeños y resolverlos de forma directa eliminando trámites internos de la computadora mientras que en quick sort quedó probado que el éxito reside en la elección del pivote pues las variantes aleatoria y de mediana de tres funcionan como un seguro de velocidad que evita que el programa se bloquee ante datos difíciles logrando que a gran escala estas pequeñas mejoras estratégicas marquen la diferencia entre una ejecución instantánea y una extremadamente lenta lo que confirma que los mejores resultados se obtienen cuando se combina la potencia de los métodos avanzados con la flexibilidad de simplificar los pasos finales del proceso.

13. Contribuciones

- Felipe Paillacar: Desarrollo de código, ejecución de pruebas experimentales y redacción del informe.
- Guillermo Vargas: Desarrollo de código, ejecución de pruebas experimentales y redacción del informe.
- Constanza Araya: Desarrollo de código, ejecución de pruebas experimentales y redacción del informe.

■ Referencias

- [1] freeCodeCamp, *Algoritmos de búsqueda: búsqueda lineal y binaria explicadas*, jun. de 2020. dirección: <https://www.freecodecamp.org/espanol/news/algoritmos-de-busqueda-lineal-y-binaria-explicadas/>.
- [2] freeCodeCamp, *Explicación del algoritmo Quickselect con ejemplos*, oct. de 2021. dirección: <https://www.freecodecamp.org/espanol/news/explicacion-del-algoritmo-quickselect-con-ejemplos/>.
- [3] freeCodeCamp, *La complejidad de los algoritmos simples y las estructuras de datos en JS*, jul. de 2022. dirección: <https://www.freecodecamp.org/espanol/news/la-complejidad-de-los-algoritmos-simples-y-las-estructuras-de-datos-en-js/>.
- [4] T. S. Engineer, *Lomuto Partition Scheme*, sep. de 2023. dirección: <https://medium.com/@strivingengineer/lomuto-partition-scheme-f20eef44e76d>.
- [5] freeCodeCamp, *Significado del algoritmo divide y vencerás*, ene. de 2023. dirección: <https://www.freecodecamp.org/espanol/news/significado-del-algoritmo-divide-y-venceras/>.
- [6] E. Valley, *La complejidad de los algoritmos*, sep. de 2023. dirección: <https://europeanvalley.es/noticias/la-complejidad-de-los-algoritmos/>.
- [7] Euroinnova, *Interpolación*, feb. de 2024. dirección: <https://tecnologia.euroinnova.com/interpolacion>.
- [8] freeCodeCamp, *Explicación de la notación Big O con ejemplos de Python y JavaScript*, feb. de 2024. dirección: <https://www.freecodecamp.org/espanol/news/explicacion-de-la-notacion-big-o-con-ejemplo/>.
- [9] GeeksforGeeks, *Interpolation Search*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/interpolation-search/>.
- [10] GeeksforGeeks, *Merge Sort*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/merge-sort/>.
- [11] GeeksforGeeks, *Quick Sort Algorithm*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>.
- [12] StuDocu, *Búsqueda exponencial: algo básico*, feb. de 2024. dirección: <https://www.studocu.com/es-mx/document/preparatoria-9-de-la-universidad-autonoma-de-nuevo-leon/tecnologia-de-la-informacion-y-comunicacion/busqueda-exponencial-algo-basico/110361390>.
- [13] Studocu, *Algoritmo recursivo de búsqueda binaria en Java: Justificación del tamaño del problema y complejidad*, <https://www.studocu.com/es/messages/question/7340543/>, Accedido el: 14 de mayo de 2026, 2024.
- [14] Tutorialspoint, *Exponential Search*, mayo de 2024. dirección: https://www.tutorialspoint.com/data_structures_algorithms/exponential_search.htm.